Workflow Comparison: FedFetch versus ECP

Anonymous Authors

***Abstract*—This note contrasts two downstream prefetching strategies for federated learning: FedFetch, which prefetches plaintext compressed model updates, and ECP, which prefetches sparse CKKS ciphertexts aggregated under a global consensus mask. We present each workflow with its native notation and highlight where they differ in communication content, client-side computation, and privacy guarantees.**

I. Notation

We deliberately keep the two symbol systems separate so that each workflow is described in its own native notation.

FedFetch symbols (following Yan *et al.* [1]):

- $w^{(t)}$: global model at the beginning of round t;
- $\hat{w}_i^{(t)}$: local model of client i at round t;
- K^t: set of clients selected for round t;
- $C_{dl}(\cdot)$: downstream compressor (masking, quantization, or low-rank);
- $\Phi_{t_1,t_2} = \sum_{\tau=t_1}^{t_2} C_{dl}\big(\Delta^{(\tau)}\big)$: accumulated compressed server updates from round t_1 to round t_2, where $\Delta^{(\tau)} = w^{(\tau+1)} - w^{(\tau)}$ is the global update at round τ;
- R: prefetch horizon—clients presampled R rounds in advance;
- P_i: prefetch start round assigned to client i.

ECP symbols:

- $G^{(t)}$: server-side global model ciphertext at the beginning of round t;
- $\mathbf{w}_i^{(t)}$: plaintext local model of client i at round t;
- $\mathcal{S}^{(t)}$: active client set at round t;
- $\mathcal{K}^{(\tau)}$: global consensus mask at round τ;
- $C_j^{(\tau)}$: aggregated CKKS ciphertext for pack j at round τ;
- $\mathcal{Q}$: server-side sliding queue of length R for aggregated update ciphertexts;
- R: prefetch horizon, with target training round $t^* = t+R$;
- $\mathcal{P}^{(t^*)}, P_i^{(t^*)}$: presampled client set for target t^* and prefetch start round;
- $\mathcal{P}^{(t)}$: current prefetch client set at round t;
- $g_i = t - t_{\text{last},i}$: number of rounds client i has missed.

II. FedFetch Workflow

FedFetch adds two phases before the standard Train phase: Prepare and Prefetch. During Prepare, the server presamples the clients that will train in round $t^* = t + R$ and assigns each client a prefetch start round $P_i \in \{t, \dots, t^* - 1\}$. During Prefetch, client i greedily downloads compressed server updates whenever it is idle.

When client i finally enters its Train phase at round t^*, its local model is synchronized by adding the base model at P_i and all compressed updates from P_i to $t^* - 1$:

$$\hat{w}_i^{(t^*)} = w^{(P_i)} + \Phi_{P_i,\,t^*-1} = w^{(P_i)} + \sum_{\tau=P_i}^{t^*-1} C_{dl}\big(\Delta^{(\tau)}\big). \quad (1)$$

After synchronization, the client runs local training and uploads its compressed update:

$$\hat{\delta}_i^{(t^*)} = C_{dl}\Big(\text{LocalTrain}\big(\hat{w}_i^{(t^*)}\big) - \hat{w}_i^{(t^*)}\Big). \quad (2)$$

Because FedFetch operates on plaintext-compressed updates, accumulation is a simple vector addition and no decryption is required at the client. The price is that the server sends model-update content in the clear (although compressed), and the communicated volume is dominated by the base model $w^{(P_i)}$ plus the accumulated update Φ_{P_i,t^*-1}.

III. ECP Workflow

ECP also separates the workflow into Prepare, Prefetch, and Train phases, but the prefetched objects are not plaintext updates. Instead, ECP prefetches sparse pack-level aggregated CKKS ciphertexts produced by the upstream ESP protocol.

A. Prepare Phase

At round t, the server selects active clients $\mathcal{S}^{(t)}$, inactive clients $\bar{\mathcal{S}}^{(t)}$, and a target-round prefetch set $\mathcal{P}^{(t^*)}$ for $t^* = t + R$. Each client $c_i \in \mathcal{P}^{(t^*)}$ is assigned a prefetch start round $P_i^{(t^*)} \in \{t, \dots, t^* - 1\}$. The server derives the current prefetch set $\mathcal{P}^{(t)} = \{c_i \mid \exists t' > t : c_i \in \mathcal{P}^{(t')},\, P_i^{(t')} \le t < t'\}$ and broadcasts $\mathcal{S}^{(t)}, \mathcal{P}^{(t)}$ to all clients. After ESP aggregation, the server obtains the sparse update ciphertext

$$C^{(t)} = \big\{C_j^{(t)}\big\}_{j \in \mathcal{K}^{(t)}}, \quad (3)$$

where each pack ciphertext has the form

$$C_j^{(t)} = \Big[\, a^{(t)} S + \tilde{p} E_j^{(t)} + \mathbf{m}_j^{(t)} \Big]_q, \quad (4)$$

with $a^{(t)} = \text{PRPG}(B, t)$ the public polynomial, $S = \sum_{i \in \mathcal{C}} s_i$ the global decryption key, and KGC compensatory ciphertexts already embedded in $C_j^{(t)}$ by ESP-Agg. The server then advances the authoritative global ciphertext

$$G^{(t+1)} = G^{(t)} - \eta C^{(t)}, \quad (5)$$

and enqueues $(t, \mathcal{K}^{(t)}, C^{(t)})$ into a sliding queue $\mathcal{Q}$ of length R, evicting entries older than $t - R + 1$. The complete ciphertext $G^{(t)}$ is never sent to clients; only the sparse queue entries are eligible for prefetch.

B. Prefetch Phase

At round t, each client $c_i \in \mathcal{P}^{(t)}$ scans $\mathcal{Q}$ and downloads only those pack ciphertexts it has missed:

$$\mathcal{L}_{i,j} \leftarrow \mathcal{L}_{i,j} \cup \big\{(\tau, C_j^{(\tau)}) \mid \tau \in (P_i^{(t^*)}, t],\ j \in \mathcal{K}^{(\tau)},\ (\tau, \cdot) \notin \mathcal{L}_{i,j}\big\}. \tag{6}$$

Non-consensus packs ($j \notin \mathcal{K}^{(\tau)}$) carry no update and are never transmitted.

C. Train Phase

At round t, each selected client $c_i \in \mathcal{S}^{(t)}$ invokes ECP-SYNC to merge the cached ciphertexts and recover the plaintext update. For each selected pack j, define the set of missed consensus rounds

$$T_{i,j} = \big\{\tau \mid (\tau, C_j^{(\tau)}) \in \mathcal{L}_{i,j}\big\}. \tag{7}$$

The client homomorphically accumulates

$$C_{\mathrm{pre},j} = \sum_{\tau \in T_{i,j}} C_j^{(\tau)} = \Big[\, A_{\mathrm{pre},j} S + \tilde{p} E_{\mathrm{pre},j} + \mathbf{m}_{\mathrm{pre},j} \Big]_q, \tag{8}$$

where $A_{\mathrm{pre},j} = \sum_{\tau \in T_{i,j}} a^{(\tau)}$ and $E_{\mathrm{pre},j} = \sum_{\tau \in T_{i,j}} E_j^{(\tau)}$. A single MK-CKKS decryption per pack then yields

$$\mathbf{m}_{\mathrm{pre},j} = \mathrm{HE.Dec}(A_{\mathrm{pre},j}, S, C_{\mathrm{pre},j}) = \Big[\big[C_{\mathrm{pre},j} - A_{\mathrm{pre},j} S\big]_q\Big]_p. \tag{9}$$

The client assembles the pack updates into $\mathbf{m}_{\mathrm{pre}}$ and synchronizes

$$\mathbf{w}_i^{(t^*)} = \mathbf{w}_i^{(t_{\mathrm{last},i})} - \eta \mathbf{m}_{\mathrm{pre}}. \tag{10}$$

IV. KEY DIFFERENCES

TABLE I
FEDFETCH VERSUS ECP

Aspect	FedFetch	ECP
Prefetched object	Plaintext compressed update	Sparse CKKS ciphertext
Server-to-client privacy	None (compressed plaintext)	HE-protected
Client accumulation	Vector addition	Homomorphic addition
Client synchronization	Direct model update	Decrypt-then-update
Storage at server	Recent global models	Queue $\mathcal{Q}$ of length R
Sparse granularity	Downstream compressor	Global consensus mask $\mathcal{K}^{(\tau)}$

The fundamental distinction is the *domain of prefetching*. FedFetch prefetches in the plaintext-compressed domain: the client downloads $w^{(P_i)}$ and Φ_{P_i, t^*-1}, adds them, and trains. ECP prefetches in the encrypted domain: the client downloads only consensus-selected pack ciphertexts, merges them homomorphically, decrypts once per pack, and then trains. Consequently, FedFetch minimizes server-side computation but exposes compressed updates to the client (and potentially to eavesdroppers), whereas ECP preserves confidentiality at the cost of one decryption per consensus pack and a sparse ciphertext prefetching protocol.

REFERENCES

[1] Q. Yan, A. Liu, S. He, M. Lecuyer, and I. Beschastnikh, "FedFetch: Faster federated learning with adaptive downstream prefetching," in *IEEE INFOCOM 2025—IEEE Conference on Computer Communications*, 2025.